

De acuerdo. Para el **Reporte 1 — EDA / Data Readiness** vamos a trabajar la población analítica como **un único dataset**, sin mencionar todavía ningún esquema de partición temporal para modelado. Tampoco usaremos “victim/non-victim”; hablaremos de **positive/event** y **negative/non-event**, porque el outcome es la primera alerta calificante observada.

La fuente principal para el EDA puede ser:

```
bu_frd.model_features_ab_final_2022_2025
```

porque ya representa la población final a nivel **party_id**, y para controles de integridad del outcome utilizaremos también:

```
bu_frd.model_cohort_behavioral_final_3m_2022_2025  
bu_frd.party_first_valid_alert_all
```

Empecemos por EDA 1 — Dataset Overview & Class Balance

Primero necesitamos congelar las métricas básicas del universo completo. Ejecuta esto en R/Spark:

```
eda_class_balance <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_records,  
    COUNT(DISTINCT party_id) AS unique_parties,  
  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,  
  
    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate,  
    AVG(CASE WHEN label_3m = 0 THEN 1.0 ELSE 0.0 END) AS negative_rate,  
  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) * 1.0 /  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS negative_to_positive_ratio  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  "  
) %>%  
  collect()  
  
eda_class_balance
```

De acuerdo con los controles que ya tenemos, deberíamos quedar alrededor de:

Metric	Expectad
Total records	688,589
Unique parties	688,589
Positive cases	62,599
Negative cases	625,990
Positive rate	9.09%
Negative rate	90.91%
Negative : Positive	10 : 1

Pero para el reporte final utilizaremos **los resultados que arroje el query**, no números escritos manualmente.

Hay una consideración importante para la narrativa del EDA: ese **9.09%** es la composición de nuestra **muestra analítica**. No debemos presentarlo como prevalencia poblacional de fraud alerts.

EDA 2 — Basic Dataset Integrity

Antes de cualquier gráfico de comportamiento, confirmemos que la unidad analítica está limpia:

```
eda_basic_integrity <- sdf_sql(
  sc,
  "
  SELECT

    COUNT(*) AS total_rows,

    COUNT(DISTINCT party_id) AS distinct_parties,

    COUNT(*) - COUNT(DISTINCT party_id) AS duplicate_party_rows,

    SUM(
      CASE
        WHEN party_id IS NULL THEN 1
        ELSE 0
      END
    ) AS null_party_id,
```

```

SUM(
  CASE
    WHEN observation_date IS NULL THEN 1
    ELSE 0
  END
) AS null_observation_date,

SUM(
  CASE
    WHEN label_3m IS NULL THEN 1
    ELSE 0
  END
) AS null_label,

SUM(
  CASE
    WHEN label_3m NOT IN (0,1) THEN 1
    ELSE 0
  END
) AS invalid_label

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_basic_integrity

Idealmente queremos:

```

duplicate_party_rows = 0
null_party_id        = 0
null_observation_date = 0
null_label           = 0
invalid_label        = 0

```

Esto se convertirá posteriormente en una pequeña sección de **Data Integrity KPI cards**, no necesariamente en una gráfica.

EDA 3 — Outcome Integrity

Este QA es más importante que el simple null check. Queremos demostrar que el label realmente respeta la definición temporal:

```

\[ label=1 \iff observation\_date < first\_alert\_date \leq
LAST\_DAY(ADD\_MONTHS(observation\_date,3)) \]

```

Ejecuta:

```
eda_label_integrity <- sdf_sql(  
  sc,  
  "  
  WITH base AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      label_3m  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
  ),  
  
  alerts AS (  
  
    SELECT  
      party_id,  
      first_alert_date  
  
    FROM bu_frd.party_first_valid_alert_all  
  
  )  
  
  SELECT  
  
    COUNT(*) AS total_rows,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date IS NULL  
        THEN 1 ELSE 0  
      END  
    ) AS positive_without_alert,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date <= b.observation_date  
        THEN 1 ELSE 0  
      END  
    ) AS positive_alert_not_future,  
  
    SUM(  
      CASE
```

```

        WHEN b.label_3m = 1
        AND a.first_alert_date >
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS positive_alert_outside_horizon,

SUM(
    CASE
        WHEN b.label_3m = 0
        AND a.first_alert_date > b.observation_date
        AND a.first_alert_date <=
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS negative_with_alert_in_horizon

FROM base b

LEFT JOIN alerts a
    ON b.party_id = a.party_id
"
) %>%
collect()

eda_label_integrity

```

Aquí idealmente todos estos errores deben ser 0.

Primera gráfica del EDA

Con esta primera sección podemos construir después una gráfica muy sencilla:

Analytic population

```

|
|—— Positive / future qualifying alert
|
|—— Negative / no qualifying alert in prediction horizon

```

Para la presentación probablemente usaría **dos KPI cards + una barra de composición**, no un pie chart, porque la diferencia 10:1 se compara mejor visualmente con barras.

También podemos incorporar un KPI:

Unique party coverage = 100%

si confirmamos:

$$TotalRecords = UniqueParties$$

Siguiente sección: Temporal Cohort Profile

Sin hablar todavía de particiones de modelado, debemos revisar cómo está distribuida la población por `observation_date`. Esto nos permite detectar concentraciones, huecos o cambios fuertes a través del tiempo.

Primero, resumen anual:

```
eda_cohort_year <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,

    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

  FROM bu_frd.model_features_ab_final_2022_2025

  GROUP BY YEAR(observation_date)

  ORDER BY observation_year
  "
) %>%
  collect()

eda_cohort_year
```

Y después a nivel mensual:

```
eda_cohort_month <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,
    MONTH(observation_date) AS observation_month,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,
```

```

SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

FROM bu_frd.model_features_ab_final_2022_2025

GROUP BY
  YEAR(observation_date),
  MONTH(observation_date)

ORDER BY
  observation_year,
  observation_month
"
) %>%
collect()

```

eda_cohort_month

Y un pequeño control del rango:

```

eda_date_range <- sdf_sql(
  sc,
  "
SELECT
  MIN(observation_date) AS min_observation_date,
  MAX(observation_date) AS max_observation_date,
  COUNT(DISTINCT observation_date) AS distinct_observation_dates

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_date_range

Con estos resultados podremos construir dos visuales posteriores:

Population volume over calendar time

Positive rate over calendar time

La segunda será particularmente importante: no queremos asumir que la tasa analítica es constante a través del tiempo.

Después de esta sección avanzaría a **Tenure / observation-history confounding**, pero primero quiero ver estos resultados temporales para no mezclar posibles efectos de calendario con tenure.

